Consumo energético de un centro de datos mediante la regla de Simpson 1/3 compuesta

Energía total que consumen los servidores durante un periodo de carga crítica de 6 horas

Estudiante: **Fernando Istaña Flores** Docente: **Jorge Luis Manrique Plasencia**

1. Contexto y datos

Un centro de datos para microservicios registra la potencia que consume el conjunto de sus servidores durante un periodo de carga crítica de 6 horas. Para dimensionar el costo del sistema de enfriamiento y de respaldo se necesita el consumo total de energía E (en kWh), que es el área bajo la curva de potencia:

$$E = \int_0^6 P(t)\,dt, \qquad P(t) = 10 + 5t\cdot\operatorname{sen}(\pi t/6) + e^{0.2t} \quad [\text{kW}], \quad t \text{ en horas}$$

La potencia combina una base de 10 kW, una componente que crece y se apaga con la carga ($5t\cdot\operatorname{sen}(\pi t/6)$, nula en t = 0 y t = 6 y máxima hacia t ≈ 4) y una tendencia exponencial suave. El seno se evalúa en **radianes**. Se pide aproximar la integral con la **regla de Simpson 1/3 compuesta con n = 6**, acotar su error y programar la regla con validación de n par y un análisis de convergencia.

2. Construcción manual — Simpson 1/3 compuesto con n = 6

Simpson 1/3 reemplaza la curva, de dos en dos subintervalos, por la parábola que pasa por los tres puntos de cada par. Sumando las áreas de esas parábolas se obtiene:

$$I \approx \frac{h}{3}\cdot\left[f(x_0) + 4\cdot\Sigma f(x \text{ impares}) + 2\cdot\Sigma f(x \text{ pares interiores}) + f(x_n) \right], \qquad h = \frac{b-a}{n}, \quad n \text{ par}$$

Los nodos impares son el centro de una parábola y pesan 4; los pares interiores son el borde común de dos parábolas y pesan 2; los extremos pesan 1. Por eso n tiene que ser par.

Paso 1 — Ancho de paso

$$h = (b-a)/n = (6-0)/6 = 1 \text{ h}$$

Paso 2 — Tabla de evaluación

i	t_i (h)	$5t_i\cdot\operatorname{sen}(\pi t_i/6)$	$e^{0.2t_i}$	$P(t_i)$ (kW)	Coef.	Coef.·$P(t_i)$
0	0	0.000000	1.000000	11.000000	1	11.000000
1	1	2.500000	1.221403	13.721403	4	54.885611
2	2	8.660254	1.491825	20.152079	2	40.304157
3	3	15.000000	1.822119	26.822119	4	107.288475
4	4	17.320508	2.225541	29.546049	2	59.092098
5	5	12.500000	2.718282	25.218282	4	100.873127
6	6	0.000000	3.320117	13.320117	1	13.320117

Valores del seno: $\operatorname{sen}(\pi/6) = 1/2$, $\operatorname{sen}(\pi/3) = \sqrt{3}/2$, $\operatorname{sen}(\pi/2) = 1$, $\operatorname{sen}(2\pi/3) = \sqrt{3}/2$, $\operatorname{sen}(5\pi/6) = 1/2$ y $\operatorname{sen}(\pi) = 0$.

Paso 3 — Aplicación de la fórmula

$$\Sigma \text{ impares} = P(1) + P(3) + P(5) = 13.721403 + 26.822119 + 25.218282 = 65.761804$$
$$\Sigma \text{ pares} = P(2) + P(4) = 20.152079 + 29.546049 = 49.698128$$
$$I \approx \frac{1}{3}\cdot[\ 11.000000 + 4(65.761804) + 2(49.698128) + 13.320117\ ]$$
$$= \frac{1}{3}\cdot[\ 11.000000 + 263.047216 + 99.396256 + 13.320117\ ]$$
$$= \frac{1}{3}\cdot(\ 386.763589\ )$$
$$= 128.921196 \approx 128.9212 \text{ kWh}$$

I_Simpson (n = 6) ≈ 128.9212 kWh

A mano, con 6 decimales, la suma de impares sale 65.761804; el programa, que no redondea, obtiene 65.761803 y 128.921195. La diferencia está en la sexta cifra decimal.

Paso 4 — Valor exacto para contrastar

La integral tiene primitiva elemental; el término 5t·sen(πt/6) se integra por partes:

$$\int t\cdot\operatorname{sen}(at)\,dt = \operatorname{sen}(at)/a^2 - t\cdot\cos(at)/a, \qquad a = \pi/6$$

$$E = \left[\, 10t + 5\left(\operatorname{sen}(at)/a^2 - t\cdot\cos(at)/a\right) + e^{0.2t}/0.2 \,\right]_0^6$$

$$= 60 + 5\cdot(0 - 6\cdot\cos(\pi)/(\pi/6)) + (e^{1.2} - 1)/0.2$$

$$= 60 + 180/\pi + (3.320117 - 1)/0.2$$

$$= 60 + 57.295780 + 11.600585 = 128.896364 \text{ kWh}$$

$$\text{Error real} = |128.896364 - 128.921195| = 0.024831 \text{ kWh} \quad (0.0193\ \%)$$

3. Cota superior del error teórico

$$|E_t| \le (b - a)\cdot h^4/180 \cdot M, \qquad M = \max |P^{(4)}(t)|, \quad t \in [0, 6]$$

Con la M de la guía (M ≈ 0.45)

$$|E_t| \le (6 - 0)\cdot(1)^4/180 \cdot 0.45 = 6/180 \cdot 0.45 = 0.015 \text{ kWh}$$

Esta cota **no es válida**: el error real (0.0248 kWh) la supera, y una cota superior nunca puede quedar por debajo del error que acota. La única explicación es que M = 0.45 no es el máximo de $|P^{(4)}(t)|$ en [0, 6].

Cálculo correcto de M

$$P^{(4)}(t) = 5\cdot\left[\, (\pi/6)^4\cdot t\cdot\operatorname{sen}(\pi t/6) - 4(\pi/6)^3\cdot\cos(\pi t/6) \,\right] + 0.2^4\cdot e^{0.2t}$$

$$= 0.375807\cdot t\cdot\operatorname{sen}(\pi t/6) - 2.870952\cdot\cos(\pi t/6) + 0.0016\cdot e^{0.2t}$$

$$P^{(4)}(0) = -2.870952 + 0.0016 = -2.869352 \quad \rightarrow \quad |P^{(4)}(0)| \approx 2.87 > 0.45$$

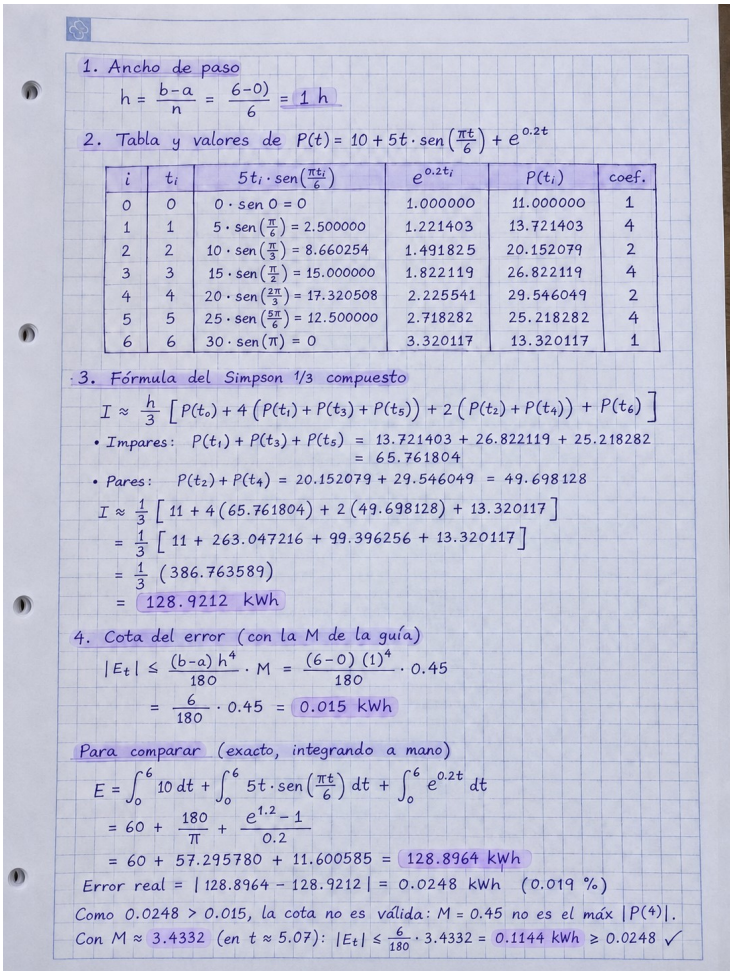
$$\max |P^{(4)}(t)| \text{ en } [0, 6] = |P^{(4)}(5.07)| \approx 3.4332$$

$$|E_t| \le 6\cdot(1)^4/180 \cdot 3.4332 = 0.1144 \text{ kWh} \quad \ge 0.0248\ \checkmark$$

Cota del error: $|E_t| \le 0.015$ kWh con la M de la guía (no válida) y $|E_t| \le 0.1144$ kWh con la M correcta. El error real de 0.0248 kWh queda por debajo de la cota corregida, como debe ser.

4. Desarrollo manual

A continuación se adjunta el desarrollo realizado a mano.



1. Ancho de paso

$$h = \frac{b-a}{n} = \frac{6-0}{6} = 1\ h$$

2. Tabla y valores de $P(t) = 10 + 5t \cdot \operatorname{sen}\left(\frac{\pi t}{6}\right) + e^{0.2t}$

i	t_i	$5t_i \cdot \operatorname{sen}\left(\frac{\pi t_i}{6}\right)$	$e^{0.2t_i}$	$P(t_i)$	coef.
0	0	$0 \cdot \operatorname{sen} 0 = 0$	1.000000	11.000000	1
1	1	$5 \cdot \operatorname{sen}\left(\frac{\pi}{6}\right) = 2.500000$	1.221403	13.721403	4
2	2	$10 \cdot \operatorname{sen}\left(\frac{\pi}{3}\right) = 8.660254$	1.491825	20.152079	2
3	3	$15 \cdot \operatorname{sen}\left(\frac{\pi}{2}\right) = 15.000000$	1.822119	26.822119	4
4	4	$20 \cdot \operatorname{sen}\left(\frac{2\pi}{3}\right) = 17.320508$	2.225541	29.546049	2
5	5	$25 \cdot \operatorname{sen}\left(\frac{5\pi}{6}\right) = 12.500000$	2.718282	25.218282	4
6	6	$30 \cdot \operatorname{sen}(\pi) = 0$	3.320117	13.320117	1

3. Fórmula del Simpson 1/3 compuesto

$$I \approx \frac{h}{3}\left[P(t_0) + 4\left(P(t_1) + P(t_3) + P(t_5)\right) + 2\left(P(t_2) + P(t_4)\right) + P(t_6)\right]$$

- Impares: $P(t_1) + P(t_3) + P(t_5) = 13.721403 + 26.822119 + 25.218282 = 65.761804$
- Pares: $P(t_2) + P(t_4) = 20.152079 + 29.546049 = 49.698128$

$$I \approx \frac{1}{3}\left[11 + 4(65.761804) + 2(49.698128) + 13.320117\right]$$
$$= \frac{1}{3}\left[11 + 263.047216 + 99.396256 + 13.320117\right]$$
$$= \frac{1}{3}(386.763589)$$
$$= 128.9212\ \text{kWh}$$

4. Cota del error (con la M de la guía)

$$|E_t| \le \frac{(b-a)\,h^4}{180} \cdot M = \frac{(6-0)\,(1)^4}{180} \cdot 0.45$$
$$= \frac{6}{180} \cdot 0.45 = 0.015\ \text{kWh}$$

Para comparar (exacto, integrando a mano)

$$E = \int_0^6 10\,dt + \int_0^6 5t \cdot \operatorname{sen}\left(\frac{\pi t}{6}\right) dt + \int_0^6 e^{0.2t}\,dt$$
$$= 60 + \frac{180}{\pi} + \frac{e^{1.2} - 1}{0.2}$$
$$= 60 + 57.295780 + 11.600585 = 128.8964\ \text{kWh}$$

Error real $= |128.8964 - 128.9212| = 0.0248$ kWh (0.019 %)

Como $0.0248 > 0.015$, la cota no es válida: $M = 0.45$ no es el máx $|P^{(4)}|$.

Con $M \approx 3.4332$ (en $t \approx 5.07$): $|E_t| \le \frac{6}{180} \cdot 3.4332 = 0.1144$ kWh ≥ 0.0248 ✓

Hoja 1: ancho de paso, tabla de evaluación con los coeficientes 1-4-2-4-2-4-1, fórmula de Simpson 1/3 compuesta, cota del error con la M de la guía, valor exacto y corrección de la cota con M ≈ 3.4332.

5. Resultado

E ≈ 128.9212 kWh con n = 6. Durante las 6 horas de carga crítica, los servidores consumen unos 128.92 kWh, frente al valor exacto de 128.8964 kWh: un error de 0.0248 kWh (0.019 %). Con el mismo n, el trapecio compuesto da 127.6200 kWh (error de 1.28 kWh), unas 50 veces peor.

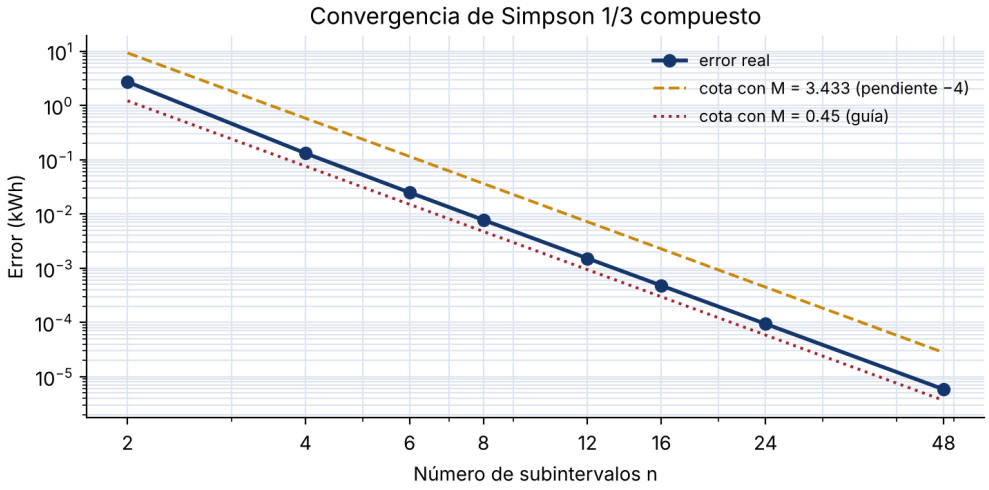
6. Aplicativo solucionador

El programa pide a, b y n; si n no es un entero par lanza una excepción propia (NImparError) y detiene la ejecución con un mensaje de error. La función simpson_13(f, a, b, n) arma el vector de coeficientes [1, 4, 2, 4, …, 2, 4, 1] con NumPy, evalúa P en toda la malla de una vez y obtiene la integral como (h/3) multiplicado por el producto escalar de los dos vectores, sin bucles. Después recorre n ∈ {2, 4, 6, 12, 24} y tabula el valor, la cota y el error real.

Análisis de convergencia

n	h (h)	I_Simpson (kWh)	$\lvert E_t\rvert$ con M = 0.45	$\lvert E_t\rvert$ con M = 3.4332	Error real
2	3	131.608592	1.215000	9.269708	2.712228
4	1.5	129.027508	0.075938	0.579357	0.131144
6	**1**	**128.921195**	**0.015000**	**0.114441**	**0.024831**
12	0.5	128.897878	9.375×10^{-4}	0.007153	0.001514
24	0.25	128.896458	5.859×10^{-5}	4.470×10^{-4}	9.405×10^{-5}

Dos cosas se leen en la tabla. Primero, el método es de **orden h^4**: al pasar de n = 12 a n = 24 (h a la mitad) el error real baja de 1.51×10^{-3} a 9.40×10^{-5}, es decir, $16.1 \approx 2^4$ veces. Segundo, en todas las filas la cota con M = 0.45 queda por debajo del error real, mientras que la cota con M = 3.4332 siempre lo cubre.



Error real y cotas en escala logarítmica: la cota de la guía (punteada roja) queda debajo del error real; la cota corregida (ámbar) lo cubre con la misma pendiente −4.

Motor de cálculo

```
"""
Análisis Numérico · Unidad 3 · Sesión 13
Integración numérica: regla de Simpson 1/3 compuesta. Energía consumida por un centro de datos.

Potencia de los servidores durante 6 horas de carga crítica (kW):
    P(t) = 10 + 5t·sen(pi·t/6) + e^(0.2·t)
Energía total (kWh):
    E = integral de 0 a 6 de P(t) dt

Fernando Istaña Flores · Universidad Peruana Unión
"""

import math
import sys

import numpy as np


# ---------------------------------------------------------------------------
# Modelo
# ---------------------------------------------------------------------------
def P(t):
    """Potencia en kW. Acepta un número o un arreglo de NumPy (el seno trabaja en radianes)."""
    return 10 + 5 * t * np.sin(np.pi * t / 6) + np.exp(0.2 * t)


def P4(t):
    """Cuarta derivada de P(t)."""
    a = np.pi / 6
    return 5 * (a**4 * t * np.sin(a * t) - 4 * a**3 * np.cos(a * t)) + 0.2**4 * np.exp(0.2 * t)


def energia_exacta(a=0.0, b=6.0):
    """Primitiva: 10t + 5[sen(at)/a² − t·cos(at)/a] + e^(0.2t)/0.2, con a = pi/6."""
    w = math.pi / 6
    F = lambda t: 10 * t + 5 * (math.sin(w * t) / w**2 - t * math.cos(w * t) / w) + math.exp(0.2 * t) /
0.2
```

```
    return F(b) - F(a)


M_GUIA = 0.45   # cota de |P''''| que da el enunciado


# ---------------------------------------------------------------------------
# Módulo de cálculo
# ---------------------------------------------------------------------------
class NImparError(ValueError):
    """n impar: Simpson 1/3 necesita que las parábolas tomen los subintervalos de dos en dos."""


def validar_n(n):
    if not isinstance(n, (int, np.integer)) or isinstance(n, bool):
        raise NImparError("n debe ser un número entero (se recibió %r)." % (n,))
    if n < 2 or n % 2 != 0:
        raise NImparError("n = %d no es válido: Simpson 1/3 necesita un número entero PAR de
subintervalos (n ≥ 2)." % n)


def coeficientes(n):
    """Vector de coeficientes 1, 4, 2, 4, …, 2, 4, 1 de longitud n + 1."""
    c = np.full(n + 1, 2.0)
    c[1::2] = 4.0
    c[0] = c[-1] = 1.0
    return c


def simpson_13(f, a, b, n):
    """
    Regla de Simpson 1/3 compuesta, vectorizada:

        h = (b − a) / n,   x = a, a + h, …, b
        I ≈ (h/3) · ( c · f(x) ),   c = [1, 4, 2, 4, …, 2, 4, 1]
    """
    validar_n(n)
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    return h / 3 * np.dot(coeficientes(n), f(x))


def cota_error(a, b, n, M):
    """|Et| ≤ (b − a)·h⁴/180 · M."""
    h = (b - a) / n
    return (b - a) * h**4 / 180 * M


# ---------------------------------------------------------------------------
# Entrada y validación
# ---------------------------------------------------------------------------
def pedir(mensaje):
    """input() que, si los datos llegan redirigidos (no desde el teclado), los repite en pantalla."""
    valor = input(mensaje)
    if not sys.stdin.isatty():
        print(valor)
    return valor


def leer_datos():
    """Pide a, b y n. Si n no es un entero par, detiene la ejecución con un mensaje de error."""
    try:
```

```
        a = float(pedir("  a (inicio del intervalo, h) = "))
        b = float(pedir("  b (fin del intervalo, h)    = "))
        texto_n = pedir("  n (subintervalos, par)      = ").strip()
    except EOFError:
        print("  (sin entrada: se usan los datos de la guía a = 0, b = 6, n = 6)")
        return 0.0, 6.0, 6
    print()
    try:
        n = int(texto_n)
    except ValueError:
        raise NImparError("n debe ser un número entero (se recibió %r)." % texto_n)
    if b <= a:
        raise ValueError("b debe ser mayor que a.")
    validar_n(n)
    return a, b, n


if __name__ == "__main__":
    print("=" * 74)
    print("ENTRADA DE DATOS")
    print("=" * 74)
    try:
        a, b, n = leer_datos()
    except ValueError as error:
        print()
        print("  ERROR: %s" % error)
        print("  Ejecución detenida.")
        sys.exit(1)
    print("  a = %g, b = %g, n = %d  (n par: válido)" % (a, b, n))

    exacta = energia_exacta(a, b)
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    c = coeficientes(n)
    y = P(x)

    print()
    print("=" * 74)
    print("PARTE 1 · SIMPSON 1/3 COMPUESTO CON n = %d" % n)
    print("=" * 74)
    print("  h = (b − a)/n = (%g − %g)/%d = %g h" % (b, a, n, h))
    print()
    print("   i     t_i    5t·sen(pi·t/6)    e^(0.2t)         P(t_i)   coef   coef·P")
    for i in range(n + 1):
        s = 5 * x[i] * math.sin(math.pi * x[i] / 6)
        print("  %2d  %6.2f   %14.6f   %10.6f   %12.6f   %4d   %10.6f" % (i, x[i], s + 0.0, math.exp(0.2
* x[i]), y[i], c[i], c[i] * y[i]))
    impares = y[1:-1:2].sum()
    pares = y[2:-1:2].sum()
    I = simpson_13(P, a, b, n)
    print()
    print("  Vector de coeficientes c = %s" % np.array2string(c.astype(int), separator=", "))
    print("  Suma de impares = %.6f     Suma de pares = %.6f" % (impares, pares))
    print("  I = (h/3)·[P(t0) + 4·%.6f + 2·%.6f + P(tn)]" % (impares, pares))
    print("    = (%g/3)·%.6f = %.6f kWh" % (h, np.dot(c, y), I))

    print()
    print("=" * 74)
    print("COTA DEL ERROR TEÓRICO")
    print("=" * 74)
    tt = np.linspace(a, b, 600001)
    M_real = float(np.max(np.abs(P4(tt))))
```

```
    t_max = float(tt[np.argmax(np.abs(P4(tt)))])
    e_real = abs(exacta - I)
    print("  |Et| ≤ (b − a)·h⁴/180 · M")
    print("  Con M = %.2f (guía):     |Et| ≤ %g·%g⁴/180·%.2f = %.6f kWh" % (M_GUIA, b - a, h, M_GUIA,
cota_error(a, b, n, M_GUIA)))
    print("  Valor exacto: E = 60 + 180/pi + (e^1.2 − 1)/0.2 = %.6f kWh" % exacta)
    print("  Error real = |%.6f − %.6f| = %.6f kWh" % (exacta, I, e_real))
    if e_real > cota_error(a, b, n, M_GUIA):
        print("  OJO: el error real supera esa cota, así que M = %.2f no es el máximo de |P''''|." %
M_GUIA)
    print("  Máximo real de |P''''| en [%g, %g]: %.6f (en t = %.4f)" % (a, b, M_real, t_max))
    print("  Con M = %.4f:          |Et| ≤ %.6f kWh  (≥ error real: correcto)" % (M_real, cota_error(a,
b, n, M_real)))

    print()
    print("=" * 74)
    print("ANÁLISIS DE CONVERGENCIA")
    print("=" * 74)
    print("  %4s %8s %14s %16s %18s %14s" % ("n", "h", "I_Simpson", "|Et| (M=0.45)", "|Et| (M=%.4f)" %
M_real, "error real"))
    filas = []
    for k in (2, 4, 6, 12, 24):
        v = simpson_13(P, a, b, k)
        filas.append((k, (b - a) / k, v, abs(exacta - v)))
        print("  %4d %8.4f %14.6f %16.6e %18.6e %14.6e" % (k, (b - a) / k, v, cota_error(a, b, k,
M_GUIA), cota_error(a, b, k, M_real), abs(exacta - v)))
    print()
    print("  Al duplicar n (12 → 24) el error real se divide entre %.1f ≈ 2⁴ = 16: orden h⁴." %
(filas[3][3] / filas[4][3]))

    print()
    print("=" * 74)
    print("PRUEBA DE LA VALIDACIÓN (n = 5)")
    print("=" * 74)
    try:
        simpson_13(P, a, b, 5)
    except NImparError as error:
        print("  ERROR: %s" % error)

    # ---------------------------------------------------------------------
    # Gráficas: P(t) con las parábolas de n = 6, y la convergencia
    # ---------------------------------------------------------------------
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        import matplotlib.ticker

        t = np.linspace(a, b, 600)
        fig, ax = plt.subplots(figsize=(7.4, 4.2), dpi=200)
        for j in range(0, n, 2):
            xs, ys = x[j:j + 3], y[j:j + 3]
            q = np.polyfit(xs, ys, 2)
            tq = np.linspace(xs[0], xs[-1], 80)
            ax.fill_between(tq, np.polyval(q, tq), color="#c8890a", alpha=0.18, linewidth=0,
                            label="área de las parábolas" if j == 0 else None)
            ax.plot(tq, np.polyval(q, tq), "--", color="#c8890a", linewidth=1.3, label="parábolas (n =
%d)" % n if j == 0 else None)
            ax.vlines(xs[0], 0, ys[0], color="#c8890a", linewidth=0.8)
        ax.vlines(x[-1], 0, y[-1], color="#c8890a", linewidth=0.8)
        ax.plot(t, P(t), color="#14366b", linewidth=2, label="P(t) = 10 + 5t·sen(πt/6) + e^(0.2t)")
        ax.scatter(x[0::2], y[0::2], color="#101c33", zorder=3, s=34, label="nodos pares")
```

```
        ax.scatter(x[1::2], y[1::2], color="#c8890a", zorder=3, s=40, label="nodos impares (coef. 4)")
        ax.set_xlabel("Tiempo t (h)")
        ax.set_ylabel("Potencia P(t) (kW)")
        ax.set_title("Simpson 1/3 compuesto, n = %d: %.4f kWh  ·  exacto: %.4f kWh" % (n, I, exacta))
        ax.set_ylim(0, max(y) * 1.15)
        ax.grid(color="#dfe5ef", linewidth=0.8)
        ax.set_axisbelow(True)
        for lado in ("top", "right"):
            ax.spines[lado].set_visible(False)
        ax.legend(frameon=False, fontsize=7.5, loc="lower center", ncol=2)
        fig.tight_layout()
        fig.savefig("simpson_s13.png")

        fig, ax = plt.subplots(figsize=(7.4, 3.8), dpi=200)
        ns = [2, 4, 6, 8, 12, 16, 24, 48]
        errs = [abs(exacta - simpson_13(P, a, b, k)) for k in ns]
        ax.loglog(ns, errs, "o-", color="#14366b", linewidth=2, label="error real")
        ax.loglog(ns, [cota_error(a, b, k, M_real) for k in ns], "--", color="#c8890a", label="cota con
M = %.3f (pendiente −4)" % M_real)
        ax.loglog(ns, [cota_error(a, b, k, M_GUIA) for k in ns], ":", color="#a32330", label="cota con M
= 0.45 (guía)")
        ax.set_xticks(ns)
        ax.xaxis.set_major_formatter(matplotlib.ticker.ScalarFormatter())
        ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
        ax.set_xlabel("Número de subintervalos n")
        ax.set_ylabel("Error (kWh)")
        ax.set_title("Convergencia de Simpson 1/3 compuesto")
        ax.grid(color="#dfe5ef", linewidth=0.8, which="both")
        for lado in ("top", "right"):
            ax.spines[lado].set_visible(False)
        ax.legend(frameon=False, fontsize=8.5)
        fig.tight_layout()
        fig.savefig("convergencia_s13.png")
        print()
        print("  Gráficas guardadas en simpson_s13.png y convergencia_s13.png")
    except ImportError:
        print()
        print("  (matplotlib no está instalado: se omitieron las gráficas)")
```

Salida del programa (a = 0, b = 6, n = 6)

```
================================================================================
ENTRADA DE DATOS
================================================================================
  a (inicio del intervalo, h) = 0
  b (fin del intervalo, h)    = 6
  n (subintervalos, par)      = 6

  a = 0, b = 6, n = 6  (n par: válido)

================================================================================
PARTE 1 · SIMPSON 1/3 COMPUESTO CON n = 6
================================================================================
  h = (b − a)/n = (6 − 0)/6 = 1 h

   i     t_i    5t·sen(pi·t/6)    e^(0.2t)         P(t_i)   coef   coef·P
   0    0.00         0.000000     1.000000      11.000000      1    11.000000
   1    1.00         2.500000     1.221403      13.721403      4    54.885611
   2    2.00         8.660254     1.491825      20.152079      2    40.304157
   3    3.00        15.000000     1.822119      26.822119      4   107.288475
   4    4.00        17.320508     2.225541      29.546049      2    59.092098
   5    5.00        12.500000     2.718282      25.218282      4   100.873127
```

```
   6    6.00         0.000000     3.320117      13.320117      1    13.320117

  Vector de coeficientes c = [1, 4, 2, 4, 2, 4, 1]
  Suma de impares = 65.761803     Suma de pares = 49.698128
  I = (h/3)·[P(t0) + 4·65.761803 + 2·49.698128 + P(tn)]
    = (1/3)·386.763586 = 128.921195 kWh

================================================================================
COTA DEL ERROR TEÓRICO
================================================================================
  |Et| ≤ (b − a)·h⁴/180 · M
  Con M = 0.45 (guía):     |Et| ≤ 6·1⁴/180·0.45 = 0.015000 kWh
  Valor exacto: E = 60 + 180/pi + (e^1.2 − 1)/0.2 = 128.896364 kWh
  Error real = |128.896364 − 128.921195| = 0.024831 kWh
  OJO: el error real supera esa cota, así que M = 0.45 no es el máximo de |P''''|.
  Máximo real de |P''''| en [0, 6]: 3.433225 (en t = 5.0688)
  Con M = 3.4332:          |Et| ≤ 0.114441 kWh  (≥ error real: correcto)

================================================================================
ANÁLISIS DE CONVERGENCIA
================================================================================
     n        h      I_Simpson    |Et| (M=0.45)    |Et| (M=3.4332)     error real
     2   3.0000     131.608592     1.215000e+00       9.269708e+00   2.712228e+00
     4   1.5000     129.027508     7.593750e-02       5.793567e-01   1.311438e-01
     6   1.0000     128.921195     1.500000e-02       1.144408e-01   2.483119e-02
    12   0.5000     128.897878     9.375000e-04       7.152552e-03   1.514013e-03
    24   0.2500     128.896458     5.859375e-05       4.470345e-04   9.404911e-05

  Al duplicar n (12 → 24) el error real se divide entre 16.1 ≈ 2⁴ = 16: orden h⁴.

================================================================================
PRUEBA DE LA VALIDACIÓN (n = 5)
================================================================================
  ERROR: n = 5 no es válido: Simpson 1/3 necesita un número entero PAR de subintervalos (n ≥ 2).

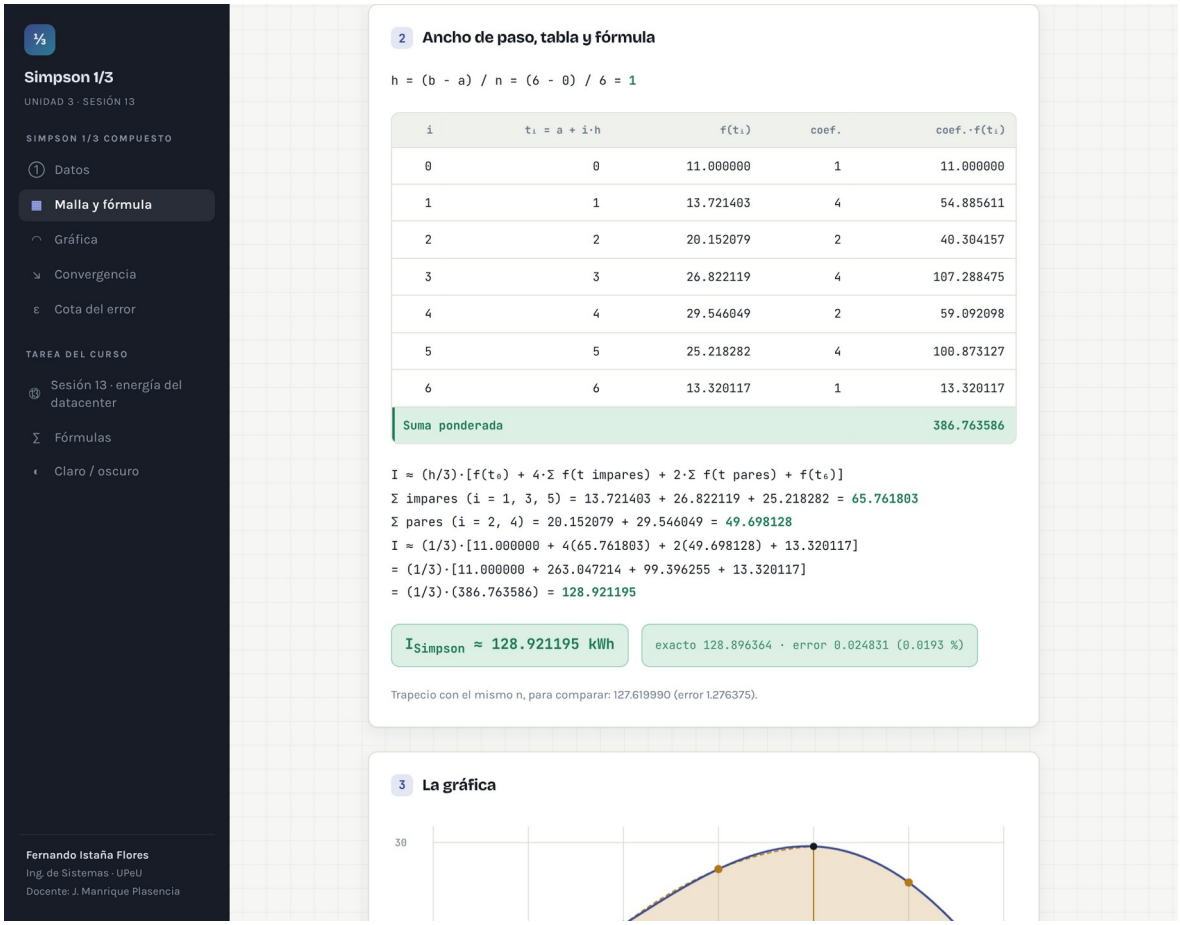
  Gráficas guardadas en simpson_s13.png y convergencia_s13.png
```

Validación con n impar (a = 0, b = 6, n = 5)

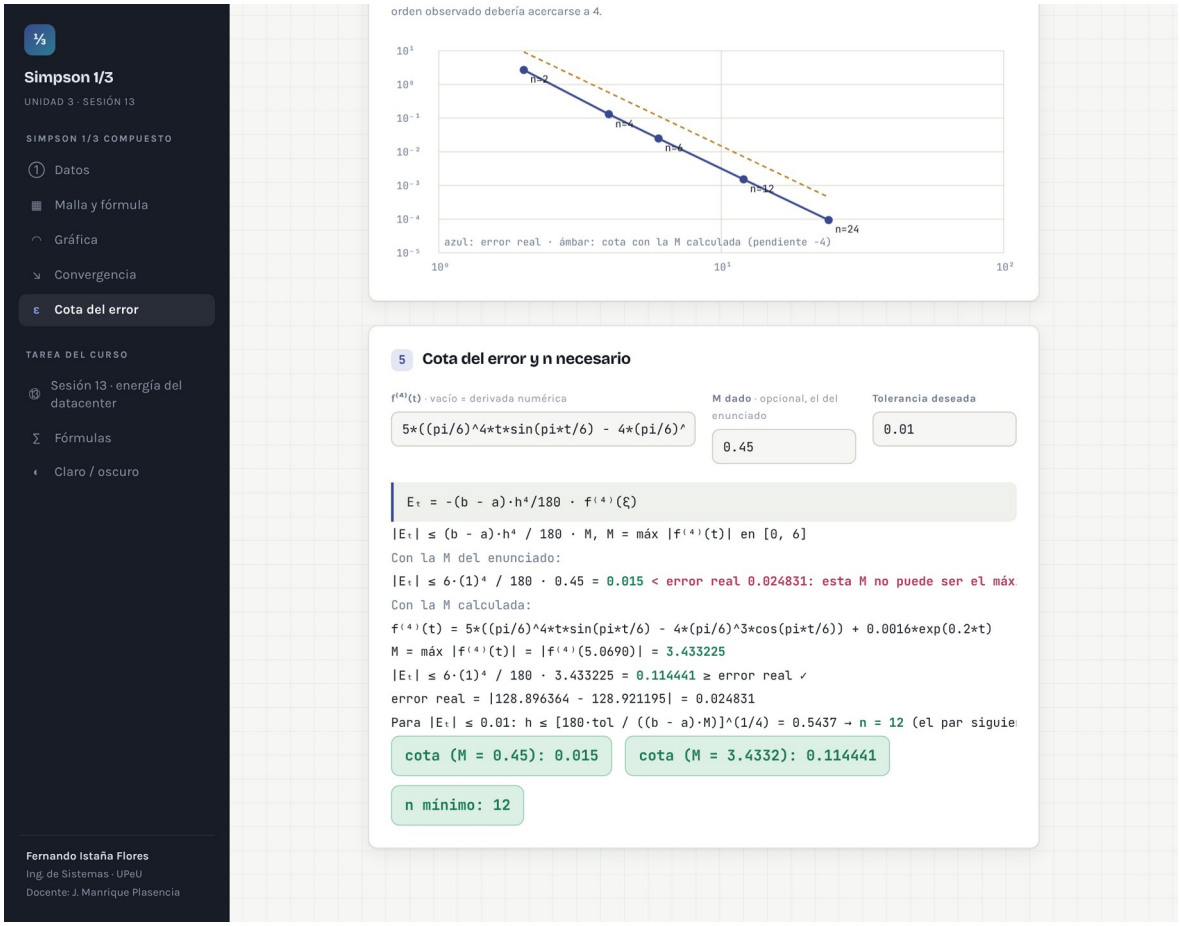
```
================================================================================
ENTRADA DE DATOS
================================================================================
  a (inicio del intervalo, h) = 0
  b (fin del intervalo, h)    = 6
  n (subintervalos, par)      = 5


  ERROR: n = 5 no es válido: Simpson 1/3 necesita un número entero PAR de subintervalos (n ≥ 2).
  Ejecución detenida.
```

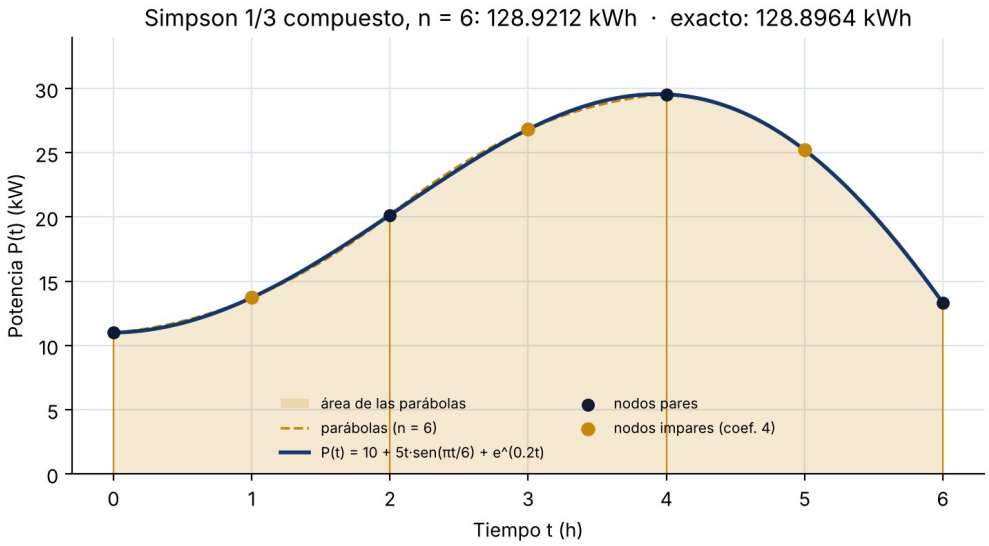
Interfaz gráfica



Aplicativo web: tabla con coeficientes, sumas de impares y pares, fórmula con los números reemplazados y resultado frente al valor exacto.



Aplicativo web: cota del error con la M del enunciado (marcada como inválida) y con la M calculada, y n mínimo para una tolerancia.



Gráfica generada por el programa: P(t), las tres parábolas de n = 6 y los nodos pares e impares.

7. Análisis de resultados

Exactitud. Con sólo 7 evaluaciones de P(t), Simpson 1/3 obtiene la energía con un error de 0.019 %. Las parábolas siguen casi exactamente a la curva (en la gráfica apenas se separan), porque P(t) es suave y su cuarta derivada es moderada.

Sobre la cota de la guía. El enunciado da M ≈ 0.45, pero la cuarta derivada vale −2.87 ya en t = 0 y alcanza $|P^{(4)}| \approx 3.43$ cerca de t = 5.07. Con M = 0.45 la cota (0.015 kWh) es menor que el error real (0.0248 kWh), lo cual es imposible para una cota superior; con la M correcta la cota es 0.1144 kWh y sí garantiza el resultado. Para el dimensionamiento conviene usar esta última: el consumo está entre 128.81 y 129.04 kWh.

Cuántos subintervalos hacen falta. Para asegurar $|E_t| \le 0.01$ kWh con M = 3.4332 se necesita $h \le [180 \cdot 0.01/(6 \cdot 3.4332)]^{(1/4)} = 0.5437$ h, es decir, n = 12 (el par siguiente a 6/0.5437 = 11.04). Con n = 12 el error real ya es de sólo 0.0015 kWh.

8. Conclusiones

Se aplicó la regla de Simpson 1/3 compuesta con n = 6 (h = 1 h) a la potencia P(t) = 10 + 5t·sen(πt/6) + e^(0.2t) y se obtuvo un consumo de 128.9212 kWh en 6 horas, frente al valor exacto de 128.8964 kWh (error de 0.0248 kWh). Se comprobó que la cota de error que resulta de la M de la guía (0.015 kWh) no es válida, y se recalculó con el máximo real de la cuarta derivada, M ≈ 3.4332, que da 0.1144 kWh. El programa valida que n sea par mediante una excepción, vectoriza los coeficientes con NumPy y muestra en la tabla de convergencia que el error cae como h^4.
